

JAVA – THREAD SYNCHRONIZATION & COMMUNICATION

Singleton ScanEngine using Double-Checked Locking

1. AIM

To implement a thread-safe Singleton ScanEngine using Double-Checked Locking and demonstrate the importance of the volatile keyword in preventing instruction reordering under the Java Memory Model. The program also considers the safer eager initialization approach.

2. ALGORITHM

1. Start the program.
2. Declare a ScanEngine reference as a private static volatile variable.
3. Create a private constructor for the ScanEngine class to prevent direct object creation.
4. Create a public getInstance() method.
5. Check whether the instance is null without acquiring the lock.
6. If the instance is null, synchronize on the ScanEngine.class object.
7. Inside the synchronized block, check again whether the instance is null.
8. If it is still null, create a new ScanEngine object.
9. Assign the newly created object to instance.
10. Return the Singleton instance.
11. Create multiple threads that request the ScanEngine instance simultaneously.
12. Verify that all threads receive the same instance.
13. Use volatile to ensure visibility and prevent unsafe instruction reordering.
14. Consider eager initialization using class loading as an alternative.
15. Stop the program.

3. PSEUDOCODE

BEGIN

CLASS ScanEngine

PRIVATE constructor

STATIC VOLATILE instance = NULL

```

METHOD getInstance()

    IF instance IS NULL THEN

        SYNCHRONIZE on ScanEngine.class

            IF instance IS NULL THEN
                instance = CREATE NEW ScanEngine
            END IF

        END SYNCHRONIZE

    END IF

    RETURN instance

END METHOD

END CLASS

CLASS Main

    METHOD main()

        CREATE multiple threads

        FOR each thread
            CALL ScanEngine.getInstance()
            DISPLAY instance reference
        END FOR

        WAIT for all threads to complete

        IF all references are the same THEN
            DISPLAY "Singleton instance verified"
        ELSE
            DISPLAY "Multiple instances detected"
        END IF

    END METHOD

END CLASS

END

```

4. JAVA PROGRAM

```

class ScanEngine {

    // volatile prevents instruction reordering
    // and guarantees visibility between threads
    private static volatile ScanEngine instance;

```

```

// Private constructor
private ScanEngine() {
    System.out.println("ScanEngine constructor executed");

    // Simulating initialization of the signature database
    try {
        Thread.sleep(100);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }

    System.out.println("Signature database initialization completed");
}

// Double-Checked Locking
public static ScanEngine getInstance() {

    if (instance == null) {

        synchronized (ScanEngine.class) {

            if (instance == null) {
                instance = new ScanEngine();
            }
        }
    }

    return instance;
}

}

public class Main {

    public static void main(String[] args) {

        Thread t1 = new Thread(() -> {
            ScanEngine engine = ScanEngine.getInstance();
            System.out.println(
                "Thread 1 received: " +
                System.identityHashCode(engine)
            );
        });

        Thread t2 = new Thread(() -> {
            ScanEngine engine = ScanEngine.getInstance();
            System.out.println(
                "Thread 2 received: " +
                System.identityHashCode(engine)
            );
        });

        Thread t3 = new Thread(() -> {
            ScanEngine engine = ScanEngine.getInstance();

```

```

        System.out.println(
            "Thread 3 received: " +
            System.identityHashCode(engine)
        );
    });

    Thread t4 = new Thread(() -> {
        ScanEngine engine = ScanEngine.getInstance();
        System.out.println(
            "Thread 4 received: " +
            System.identityHashCode(engine)
        );
    });

    // Start all threads
    t1.start();
    t2.start();
    t3.start();
    t4.start();

    try {
        t1.join();
        t2.join();
        t3.join();
        t4.join();
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }

    // Verify singleton
    ScanEngine a = ScanEngine.getInstance();
    ScanEngine b = ScanEngine.getInstance();

    if (a == b) {
        System.out.println("Singleton instance verified successfully.");
    } else {
        System.out.println("Multiple instances detected.");
    }
}
}

```

5. IMPLEMENTATION

The program is implemented in Java using the Double-Checked Locking Singleton pattern. The `ScanEngine` constructor is private, allowing only one instance of the class to be created. The `getInstance()` method first checks whether the instance exists without synchronization. If the instance is not available, synchronization is applied and the condition is checked again before creating the object.

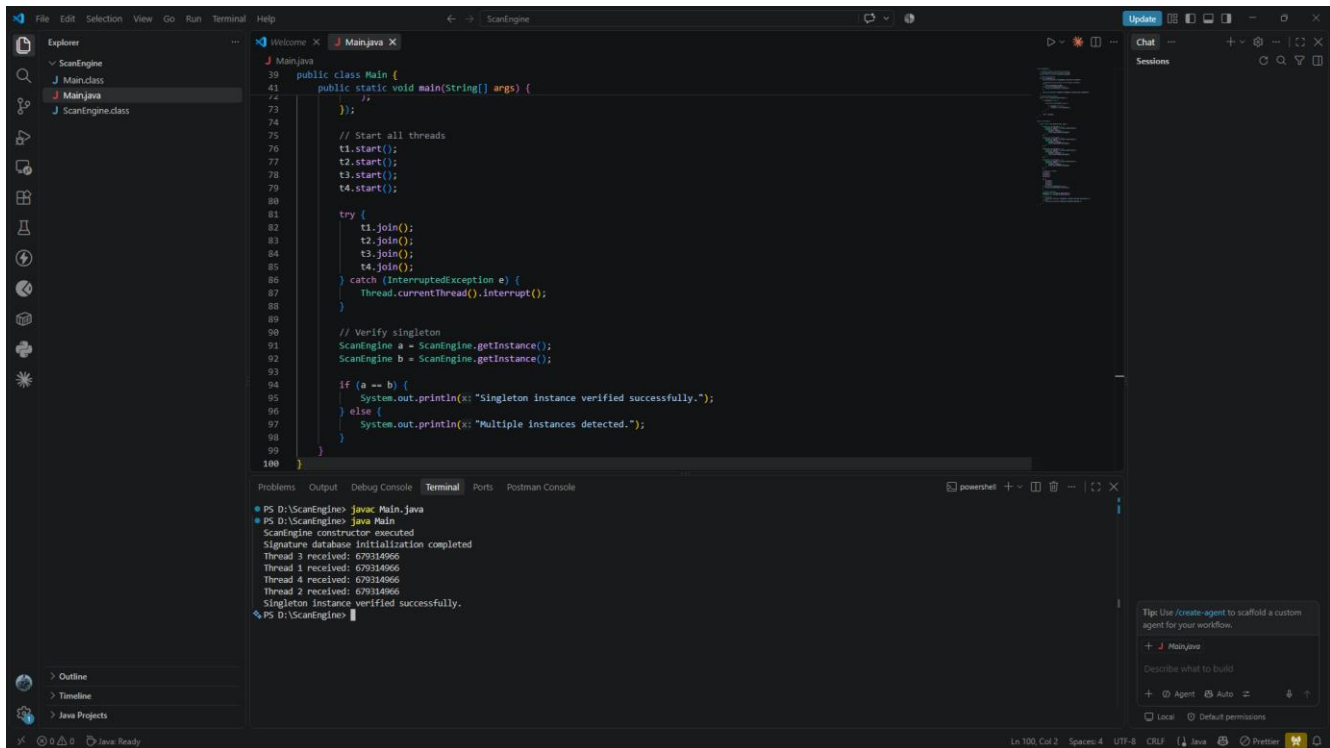
The instance variable is declared using the `volatile` keyword. This is essential because object creation is not necessarily a single atomic operation. It can conceptually involve memory allocation, object initialization, and assignment of the reference. Without `volatile`, the Java Memory Model can permit

unsafe reordering, allowing another thread to observe a non-null reference before initialization has completely finished.

The volatile keyword provides the required visibility and ordering guarantees, preventing unsafe publication of a partially initialized ScanEngine object.

6. OUTPUT

The output is to be obtained by executing the program in VS Code. The identity hash-code value may differ between executions.



The screenshot displays the Visual Studio Code editor with a Java project named 'ScanEngine'. The 'Main.java' file is open, showing a public class 'Main' with a 'main' method. The code starts by starting four threads (t1, t2, t3, t4), each calling 'ScanEngine.getInstance()'. These threads are then joined. After joining, the code verifies the singleton by calling 'ScanEngine.getInstance()' again and comparing it with the first instance. If they are the same, it prints 'Singleton instance verified successfully.'; otherwise, it prints 'Multiple instances detected.'.

```
1  Main.java
2  public class Main {
3      public static void main(String[] args) {
4          //
5          // Start all threads
6          t1.start();
7          t2.start();
8          t3.start();
9          t4.start();
10
11      try {
12          t1.join();
13          t2.join();
14          t3.join();
15          t4.join();
16      } catch (InterruptedException e) {
17          Thread.currentThread().interrupt();
18      }
19
20      // Verify singleton
21      ScanEngine a = ScanEngine.getInstance();
22      ScanEngine b = ScanEngine.getInstance();
23
24      if (a == b) {
25          System.out.println("Singleton instance verified successfully.");
26      } else {
27          System.out.println("Multiple instances detected.");
28      }
29  }
```

The terminal output shows the execution of the program. It starts with 'PS D:\ScanEngine> javac Main.java' and 'PS D:\ScanEngine> java Main'. The output indicates that the 'ScanEngine' constructor was executed, the signature database initialization was completed, and four threads (Thread 1, 2, 3, 4) were received. Finally, it prints 'Singleton instance verified successfully.'.

```
PS D:\ScanEngine> javac Main.java
PS D:\ScanEngine> java Main
ScanEngine constructor executed
Signature database initialization completed
Thread 1 received: 679314066
Thread 2 received: 679314066
Thread 3 received: 679314066
Thread 4 received: 679314066
Singleton instance verified successfully.
PS D:\ScanEngine>
```

7. RESULT

Thus, the thread-safe Singleton ScanEngine was successfully implemented using Double-Checked Locking. The use of the volatile keyword prevents unsafe instruction reordering and ensures that all threads observe a completely initialized ScanEngine object. The program also verifies that only a single instance is created and shared among multiple threads.